

# Hand Detection

## 1. Experiment Objective

Use MediaPipe Hands to detect the 21 3D hand keypoints in the camera image, with a dual-panel display (original image + skeleton pose).

## 2. Experiment Principle

### 1. Key Terms

| Keyword              | Description                                                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| MediaPipe Hands      | Google's hand keypoint detection model that predicts 21 3D hand joint coordinates from a single frame, supporting one-hand and two-hand modes   |
| Hand keypoints       | 21 predefined joint coordinate points, from the wrist to the fingertips, covering the palm and all five fingers                                 |
| Dual-panel display   | Left: original image with the hand skeleton overlaid; right: skeleton pose on a pure black background, making keypoint accuracy easy to inspect |
| Detection confidence | The minimum confidence threshold for MediaPipe's initial detection stage (default 0.5)                                                          |
| Tracking confidence  | The minimum confidence threshold for MediaPipe's tracking stage (default 0.5)                                                                   |
| Static image mode    | True: detects each frame independently; False (default): enables cross-frame tracking, suitable for video streams                               |

### 2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start hand detection
```

**Data flow:** ESP32 camera → `/camera/image_raw` → image conversion → MediaPipe Hands inference (21 keypoints + finger skeleton) → dual-panel display (left = original image overlay, right = skeleton on black background) → real-time display via `cv2.imshow`.

## 3. Experiment Steps

### 1. Hardware Preparation

| Hardware             | Requirement  |
|----------------------|--------------|
| PTZ version          | ☑ Supported  |
| No-PTZ version       | ☑ Supported  |
| Required peripherals | ESP32 camera |

## 2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

## 3. Start hand detection (Terminal 2)

```
ros2 launch microros_v2_mediapipe_pkg hand_detector.launch.py
```

## 4. Operating Instructions

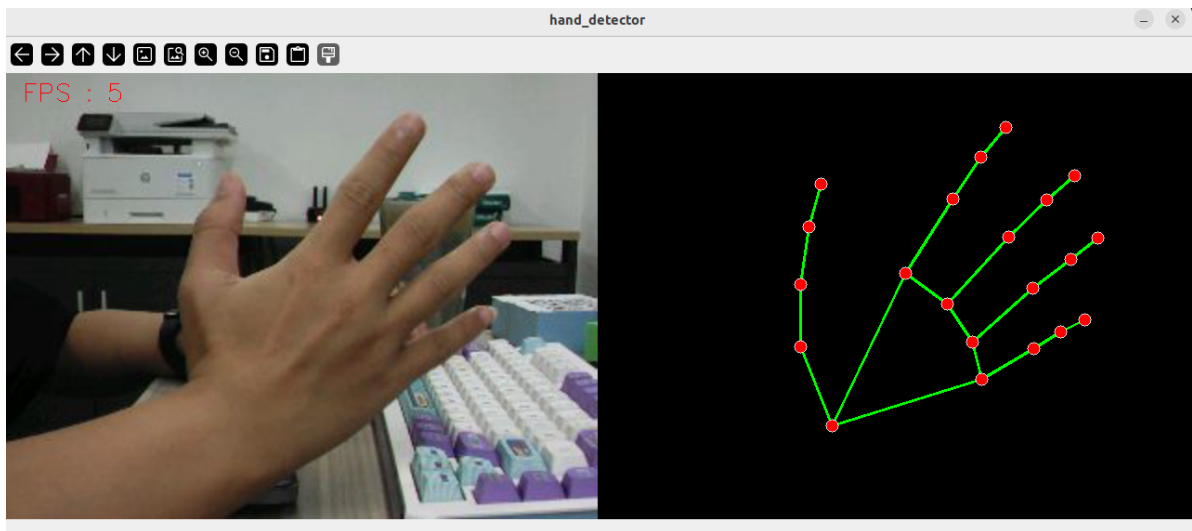
Real-time dual-panel display: the left panel shows the original image with the hand skeleton overlaid (21 red dots + connecting lines), and the right panel shows the skeleton pose on a black background. Press **q** to exit.

## 5. How to Stop

Press **Ctrl+C** in each terminal, one by one, to stop.

## 4. Observed Results

- Put your palm in the camera's field of view: 21 keypoints are marked with red dots and the finger skeleton is drawn with colored connecting lines
- In two-hand mode, both hands are detected simultaneously, each annotated independently
- The right panel shows the skeleton on a pure black background, making keypoint position accuracy easy to inspect



## 5. Code Walkthrough

Source code paths:

- Launch:

```
~/microros_ws/src/microros_v2_mediapipe_pkg/launch/hand_detector.launch.py
```

- Node:

```
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/HandDetector.py
```

### 1. Core Node Analysis

`HandDetector` uses MediaPipe Hands to detect the 21 hand keypoints and visualizes them in a dual-panel display.

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/HandDetector.py

class HandDetector(Node):
    def __init__(self):
        super().__init__("hand_detector_node")
        self.declare_parameter("rgb_topic", "/camera/image_raw")
        self.declare_parameter("static_image_mode", False)
        self.declare_parameter("max_hands", 2)
        self.declare_parameter("detector_con", 0.5)
        self.declare_parameter("track_con", 0.5)
        self.declare_parameter("draw_on_source", False)

        # MediaPipe Hands initialization
        self.mp_hand = mp.solutions.hands
        self.hands = self.mp_hand.Hands(
            static_image_mode=static_image_mode,
            max_num_hands=max_hands,
            min_detection_confidence=detector_con,
            min_tracking_confidence=track_con)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        frame_rgb = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
        results = self.hands.process(frame_rgb)
```

```
# Dual panel: left = original image with skeleton overlaid, right =
skeleton on pure black background
black_board = np.zeros(frame.shape, dtype=np.uint8)
if results.multi_hand_landmarks:
    for hand_landmarks in results.multi_hand_landmarks:
        self.mp_draw.draw_landmarks(frame, hand_landmarks,
            self.mp_hand.HAND_CONNECTIONS, lm_draw_spec, conn_draw_spec)
        self.mp_draw.draw_landmarks(black_board, hand_landmarks,
            self.mp_hand.HAND_CONNECTIONS)
    combined = np.hstack((frame, black_board))
    cv.imshow(self.window_name, combined)
```

Key logic:

- `MEDIAPIPE_DISABLE_TF/GPU`: the Ubuntu VM has no GPU, so the TensorFlow/GPU backend is disabled to avoid startup errors
- `mediapipe.solutions.hands.Hands()`: core detector; `max_num_hands=2` supports two hands; `min_detection_confidence=0.5` filters low-confidence detections
- `static_image_mode=False` (default): enables cross-frame tracking, reducing computation, suitable for video streams
- Dual panel via `np.hstack`: skeleton-overlaid original image + skeleton on a pure black background shown side by side, helping to inspect keypoint position accuracy

## 2. Key Parameters

Parameter definition files:

- Launch:  
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/hand_detector.launch.py`
- Node:  
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/HandDetector.py`

| Parameter                      | Type  | Default | Description                                                             |
|--------------------------------|-------|---------|-------------------------------------------------------------------------|
| <code>static_image_mode</code> | bool  | false   | True=detect each frame independently, False=enable cross-frame tracking |
| <code>max_hands</code>         | int   | 2       | Maximum number of hands to detect                                       |
| <code>detector_con</code>      | float | 0.5     | Detection confidence threshold                                          |
| <code>track_con</code>         | float | 0.5     | Tracking confidence threshold                                           |
| <code>draw_on_source</code>    | bool  | false   | True=overlay on the original image, False=dual-panel mode               |

## 3. Node Description

| Node                            | Function                                                      |
|---------------------------------|---------------------------------------------------------------|
| <code>hand_detector_node</code> | MediaPipe Hands 21-point detection + dual-panel visualization |

## 6. Frequently Asked Questions

| Issue                                   | Cause                                                        | Solution                                                     |
|-----------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------|
| Hand not detected                       | Insufficient lighting or the hand is too far from the camera | Improve the lighting and bring the hand closer to the camera |
| Only one hand detected in two-hand mode | <code>max_hands</code> is not set to 2                       | Set <code>max_hands:=2</code>                                |